After a developer pushes new code to the mainline, the commit stage starts by checking out the source code from the repository. The starting point is always a commit to the main branch. Following continuous integration practices, we'll aim to work in small steps and integrate our changes with the main branch multiple times a day (*continuously*).

Next, the pipeline can perform several types of static code analysis. For this example, we'll focus on vulnerability scanning. In a real-world project, you would probably want to include additional steps, such as running static code analysis to identify security issues and check compliance with specific coding standards (*code linting*).

Finally, the pipeline builds the application and runs automated tests. In the commit stage, we include technically focused tests that don't require deploying the entire application. These are unit tests and often integration tests. If the integration tests take too long, it's better to move them to the acceptance stage to keep the commit stage fast.

The vulnerability scanner we'll use in the Polar Bookshop project is `grype` (https://github.com/anchore/grype), a powerful open source tool that's increasingly used in the cloud native world. For example, it's part of the supply chain security solution provided by the VMware Tanzu Application Platform. You can find instructions on how to install it in section A.4 of appendix A.

Let's see how grype works. Open a Terminal window, navigate to the root folder of your Catalog Service project (catalog-service), and build the application with `./gradlew build`. Then use `grype` to scan your Java codebase for vulnerabilities. The tool will download a list of known vulnerabilities (a *vulnerability database*) and scan your project against them. The scanning happens locally on your machine, which means none of your